<copy-cw-envt.py>

```
import pybullet as p
import pybullet_data
import time
import numpy as np
import random
import creature
import creature_with_fixed_body
import simulation
import population
import population_with_fixed_body
import genome
import genome_with_fixed_body
import os

# p.connect(p.GUI)
# p.setAdditionalSearchPath(pybullet_data.getDataPath())

import random
#import pybullet as p
import math
#import pybullet as p



def make_mountain(num_rocks=100, max_size=0.25, arena_size=10, mountain_height=5):
    def gaussian(x, y, sigma=arena_size/4):
        """Return the height of the mountain at position (x, y) using a Gaussian function."""
        return mountain_height * math.exp(-((x**2 + y**2) / (2 * sigma**2)))
```

```
    for _ in range(num_rocks):
        x = random.uniform(-1 * arena_size/2, arena_size/2)
        y = random.uniform(-1 * arena_size/2, arena_size/2)
        z = gaussian(x, y)  # Height determined by the Gaussian function

        # Adjust the size of the rocks based on height. Higher rocks (closer to the peak) will be smaller.
        size_factor = 1 - (z / mountain_height)
        size = random.uniform(0.1, max_size) * size_factor

        orientation = p.getQuaternionFromEuler([random.uniform(0, 3.14), random.uniform(0, 3.14), random.uniform(0, 3.14)])
        rock_shape = p.createCollisionShape(p.GEOM_BOX, halfExtents=[size, size, size])
        rock_visual = p.createVisualShape(p.GEOM_BOX, halfExtents=[size, size, size], rgbaColor=[0.5, 0.5, 0.5, 1])
        rock_body = p.createMultiBody(baseMass=0, baseCollisionShapeIndex=rock_shape, baseVisualShapeIndex=rock_visual, basePosition=[x, y, z], baseOrientation=orientation)



def make_rocks(num_rocks=100, max_size=0.25, arena_size=10):
    for _ in range(num_rocks):
        x = random.uniform(-1 * arena_size/2, arena_size/2)
        y = random.uniform(-1 * arena_size/2, arena_size/2)
        z = 0.5  # Adjust based on your needs
        size = random.uniform(0.1,max_size)
        orientation = p.getQuaternionFromEuler([random.uniform(0, 3.14), random.uniform(0, 3.14), random.uniform(0, 3.14)])
        rock_shape = p.createCollisionShape(p.GEOM_BOX, halfExtents=[size, size, size])
```

```
        rock_visual = p.createVisualShape(p.GEOM_BOX, halfExtents=[size, size, size], rgbaColor=[0.5, 0.5, 0.5, 1])
        rock_body = p.createMultiBody(baseMass=0, baseCollisionShapeIndex=rock_shape, baseVisualShapeIndex=rock_visual, basePosition=[x, y, z], baseOrientation=orientation)


def make_arena(arena_size=10, wall_height=1):
    wall_thickness = 0.5
    floor_collision_shape = p.createCollisionShape(shapeType=p.GEOM_BOX, halfExtents=[arena_size/2, arena_size/2, wall_thickness])
    floor_visual_shape = p.createVisualShape(shapeType=p.GEOM_BOX, halfExtents=[arena_size/2, arena_size/2, wall_thickness], rgbaColor=[1, 1, 0, 1])
    floor_body = p.createMultiBody(baseMass=0, baseCollisionShapeIndex=floor_collision_shape, baseVisualShapeIndex=floor_visual_shape, basePosition=[0, 0, -wall_thickness])

    wall_collision_shape = p.createCollisionShape(shapeType=p.GEOM_BOX, halfExtents=[arena_size/2, wall_thickness/2, wall_height/2])
    wall_visual_shape = p.createVisualShape(shapeType=p.GEOM_BOX, halfExtents=[arena_size/2, wall_thickness/2, wall_height/2], rgbaColor=[0.7, 0.7, 0.7, 1])  # Gray walls

    # Create four walls
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape, baseVisualShapeIndex=wall_visual_shape, basePosition=[0, arena_size/2, wall_height/2])
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape, baseVisualShapeIndex=wall_visual_shape, basePosition=[0, -arena_size/2, wall_height/2])

    wall_collision_shape = p.createCollisionShape(shapeType=p.GEOM_BOX, halfExtents=[wall_thickness/2, arena_size/2, wall_height/2])
    wall_visual_shape = p.createVisualShape(shapeType=p.GEOM_BOX, halfExtents=[wall_thickness/2, arena_size/2, wall_height/2], rgbaColor=[0.7, 0.7, 0.7, 1])  # Gray walls

    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape,
```

```
baseVisualShapeIndex=wall_visual_shape, basePosition=[arena_size/2, 0, wall_height/2])
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape, baseVisualShapeIndex=wall_visual_shape, basePosition=[-arena_size/2, 0, wall_height/2])


def default_simple_example() :
    p.connect(p.GUI)
    p.setAdditionalSearchPath(pybullet_data.getDataPath())

    p.setGravity(0, 0, -10)

    arena_size = 20
    make_arena(arena_size=arena_size)

    #make_rocks(arena_size=arena_size)

    mountain_position = (0, 0, -1)  # Adjust as needed
    mountain_orientation = p.getQuaternionFromEuler((0, 0, 0))
    p.setAdditionalSearchPath('shapes/')
    # mountain = p.loadURDF("mountain.urdf", mountain_position, mountain_orientation, useFixedBase=1)
    # mountain = p.loadURDF("mountain_with_cubes.urdf", mountain_position, mountain_orientation, useFixedBase=1)

    mountain = p.loadURDF("gaussian_pyramid.urdf", mountain_position, mountain_orientation, useFixedBase=1)

    # generate a random creature
    cr = creature.Creature(gene_count=3)
    # save it to XML
```

```
    with open('test.urdf', 'w') as f:
        f.write(cr.to_xml())
    # load it into the sim
    rob1 = p.loadURDF('test.urdf', (0, 0, 10))

    p.setRealTimeSimulation(1)

def run_the_simul_in_the_arena(connect_mode, cr):
    # generate a random creature
#     cr = creature.Creature(gene_count=3)
    sim = simulation.Simulation()
    if p.GUI == connect_mode:
        sim.set_as_GUI()
        print("run as GUI Mode")
    else:
        print("run as DIRECT Mode")
    sim.run_one_creature_in_the_arena(cr, 2400 * 100)

#run genetic algorithm without threads
def GA_without_threads(population_size=10, gene_count=3, generation_count=1000, motor_update_count=2400):
    print(f'population_size({population_size}), gene_count({gene_count}), generation_count({generation_count}), motor_update_count({motor_update_count})')
    pop = population.Population(pop_size=population_size,
                                gene_count=gene_count)
    #sim = simulation.ThreadedSim(pool_size=1)
    sim = simulation.Simulation()

    for iteration in range(generation_count):
```

```
            # this is a non-threaded version
            # where we just call run_creature instead
            # of eval_population
            for cr in pop.creatures:
                sim.run_one_creature_in_the_arena(cr=cr, motor_update_count=motor_update_count)
#                 sim.run_creature(cr, 2400)
            #sim.eval_population(pop, 2400)
            fits = [cr.get_distance_travelled()
                    for cr in pop.creatures]
            links = [len(cr.get_expanded_links())
                    for cr in pop.creatures]
            if iteration % 100 == 0 or iteration == generation_count -1:
                print(iteration, "fittest:", np.round(np.max(fits), 3),
                      "mean:", np.round(np.mean(fits), 3), "mean links", np.round(np.mean(links)), "max links",
np.round(np.max(links)))
            fit_map = population.Population.get_fitness_map(fits)
            new_creatures = []
            for i in range(len(pop.creatures)):
                p1_ind = population.Population.select_parent(fit_map)
                p2_ind = population.Population.select_parent(fit_map)
                p1 = pop.creatures[p1_ind]
                p2 = pop.creatures[p2_ind]
                # now we have the parents!
                dna = genome.Genome.crossover(p1.dna, p2.dna)
                dna = genome.Genome.point_mutate(dna, rate=0.1, amount=0.25)
                dna = genome.Genome.shrink_mutate(dna, rate=0.25)
                dna = genome.Genome.grow_mutate(dna, rate=0.1)
                cr = creature.Creature(1)
                cr.update_dna(dna)
```

```
                new_creatures.append(cr)
        DIRNAME_CSVS = 'csvs'
        if not os.path.isdir(DIRNAME_CSVS):
            os.mkdir(DIRNAME_CSVS)
        # elitism
        max_fit = np.max(fits)
        for cr in pop.creatures:
            if cr.get_distance_travelled() == max_fit:
                new_cr = creature.Creature(1)
                new_cr.update_dna(cr.dna)
                new_creatures[0] = new_cr
                filename = os.path.join(DIRNAME_CSVS, "elite_"+ "P" + str(population_size) + "_" + "GC"
+ str(gene_count)    + "_" + "IT" + str(iteration).zfill(5)+"_" + str(np.round(max_fit, 3))    +".csv")
                genome.Genome.to_csv(cr.dna, filename)
                break

        pop.creatures = new_creatures


#run Genetic algorithm with encoding scheme
def          GA_without_threads2(population_size=10,          gene_count=3,          generation_count=1000,
motor_update_count=2400):
    print(f'population_size({population_size}),                                    gene_count({gene_count}),
generation_count({generation_count}), motor_update_count({motor_update_count})')
    pop = population_with_fixed_body.Population_with_fixed_body(pop_size=population_size,
                                    gene_count=gene_count)
    #sim = simulation.ThreadedSim(pool_size=1)
    sim = simulation.Simulation()

    for iteration in range(generation_count):
```

```
            # this is a non-threaded version
            # where we just call run_creature instead
            # of eval_population
            for cr in pop.creatures:
                sim.run_one_creature_in_the_arena(cr=cr, motor_update_count=motor_update_count)
#                 sim.run_creature(cr, 2400)
            #sim.eval_population(pop, 2400)
            fits = [cr.get_distance_travelled()
                    for cr in pop.creatures]
            links = [len(cr.get_flat_links2())
                    for cr in pop.creatures]
            if iteration % 100 == 0 or iteration == generation_count -1:
                print(iteration, "fittest:", np.round(np.max(fits), 3),
                      "mean:", np.round(np.mean(fits), 3), "mean links", np.round(np.mean(links)), "max links",
np.round(np.max(links)))
            fit_map = population_with_fixed_body.Population_with_fixed_body.get_fitness_map2(fits)
            new_creatures = []
            for i in range(len(pop.creatures)):
                p1_ind = population_with_fixed_body.Population_with_fixed_body.select_parent2(fit_map)
                p2_ind = population_with_fixed_body.Population_with_fixed_body.select_parent2(fit_map)
                p1 = pop.creatures[p1_ind]
                p2 = pop.creatures[p2_ind]
                # now we have the parents!
                dna = genome_with_fixed_body.Genome_with_fixed_body.crossover2(p1.dna, p2.dna)
                dna = genome_with_fixed_body.Genome_with_fixed_body.point_mutate2(dna, rate=0.1,
amount=0.25)
                dna = genome_with_fixed_body.Genome_with_fixed_body.shrink_mutate2(dna, rate=0.25)
                dna = genome_with_fixed_body.Genome_with_fixed_body.grow_mutate2(dna, rate=0.1)
                cr = creature_with_fixed_body.Creature_with_fixed_body(1)
```

```
                cr.update_dna2(dna)
                new_creatures.append(cr)
        DIRNAME_CSVS = 'csvs'
        if not os.path.isdir(DIRNAME_CSVS):
            os.mkdir(DIRNAME_CSVS)
        # elitism
        max_fit = np.max(fits)
        for cr in pop.creatures:
            if cr.get_distance_travelled() == max_fit:
                new_cr = creature_with_fixed_body.Creature_with_fixed_body(1)
                new_cr.update_dna2(cr.dna)
                new_creatures[0] = new_cr
                filename = os.path.join(DIRNAME_CSVS, "elite_FB_"+ "P" + str(population_size) + "_" +
"GC" + str(gene_count)   + "_" + "IT" + str(iteration).zfill(5)+"_" + str(np.round(max_fit, 3))   +".csv")
                genome_with_fixed_body.Genome_with_fixed_body.to_csv2(cr.dna, filename)
                break

        pop.creatures = new_creatures

#run genetic algorithm with threads in linux
def GA_with_threads(population_size=10, gene_count=3, generation_count=1000,
motor_update_count=2400):

    pop = population.Population(pop_size=population_size,
                                gene_count=gene_count)
    sim = simulation.ThreadedSim(pool_size=1)
    #sim = simulation.Simulation()

    for iteration in range(generation_count):
```

```
        sim.eval_population_in_the_arena(pop, motor_update_count)
        fits = [cr.get_distance_travelled()
                for cr in pop.creatures]
        links = [len(cr.get_expanded_links())
                for cr in pop.creatures]
        if iteration % 100 == 0 or iteration == generation_count -1:
            print(iteration, "fittest:", np.round(np.max(fits), 3),
                "mean:", np.round(np.mean(fits), 3), "mean links", np.round(np.mean(links)), "max links",
np.round(np.max(links)), population_size, gene_count)
        fit_map = population.Population.get_fitness_map(fits)
        new_creatures = []
        for i in range(len(pop.creatures)):
            p1_ind = population.Population.select_parent(fit_map)
            p2_ind = population.Population.select_parent(fit_map)
            p1 = pop.creatures[p1_ind]
            p2 = pop.creatures[p2_ind]
            # now we have the parents!
            dna = genome.Genome.crossover(p1.dna, p2.dna)
            dna = genome.Genome.point_mutate(dna, rate=0.1, amount=0.25)
            dna = genome.Genome.shrink_mutate(dna, rate=0.25)
            dna = genome.Genome.grow_mutate(dna, rate=0.1)
            cr = creature.Creature(1)
            cr.update_dna(dna)
            new_creatures.append(cr)

        DIRNAME_CSVS = 'csvs'
        if not os.path.isdir(DIRNAME_CSVS):
            os.mkdir(DIRNAME_CSVS)
```

```
            # elitism
            max_fit = np.max(fits)
            for cr in pop.creatures:
                if cr.get_distance_travelled() == max_fit:
                    new_cr = creature.Creature(1)
                    new_cr.update_dna(cr.dna)
                    new_creatures[0] = new_cr
                    filename = os.path.join(DIRNAME_CSVS, "elite_"+ "P" + str(population_size) + "_" + "GC" + str(gene_count)    + "_" + "IT" + str(iteration).zfill(5)+"_" + str(np.round(max_fit, 3))    +".csv")
                    genome.Genome.to_csv(cr.dna, filename)
                    break

            pop.creatures = new_creatures


if __name__ == '__main__':

    # generate a random creature
    cr = creature.Creature(gene_count=1)
#     dna = genome.Genome.from_csv(os.path.join("csvs", "elite_P20_GC5_IT00019_13.201.csv"))
    dna = genome.Genome.from_csv(os.path.join(".", "elite_P20_GC3_IT02999_15.323.csv"))
    cr.update_dna(dna)
    run_the_simul_in_the_arena(p.GUI, cr)



#                 GA_without_threads2(population_size=10,     gene_count=4,     generation_count=400, motor_update_count=2400)
```

```
#       for ps in [10, 20, 30, 40, 50]:
        for ps in [10, 20,]:
            # for gc in [3, 5, 7, 9]:
            for gc in [3, 5]:
                for genc in [10, 20]:
                    GA_without_threads(population_size=ps,     gene_count=gc,     generation_count=genc,
motor_update_count=2400)
#       run_the_simul(p.DIRECT)
        run_the_simul_in_the_arena(p.GUI, creature.Creature(gene_count=3))
#       default_simple_example()
```

<creature_with_fixed_body>

```
import genome
import genome_with_fixed_body
from xml.dom.minidom import getDOMImplementation
from enum import Enum
import numpy as np
import creature

#class with encoding scheme
class Creature_with_fixed_body:
    def __init__(self, gene_count):
#spec and dna is from genome_with_fixed_body class
        self.spec = genome_with_fixed_body.Genome_with_fixed_body.get_gene_spec2()
        self.dna = genome_with_fixed_body.Genome_with_fixed_body.get_random_genome2(gene_count)
        self.flat_links = None
        #self.exp_links = None
```

```
        self.motors = None
        self.start_position = None
        self.last_position = None

    def get_flat_links2(self):
        if self.flat_links == None:
            gdicts = genome_with_fixed_body.Genome_with_fixed_body.get_genome_dicts2(self.dna, self.spec)
            self.flat_links = genome_with_fixed_body.Genome_with_fixed_body.genome_to_flat_links2(gdicts)
        return self.flat_links

    def to_xml(self):
        self.get_flat_links2()
        domimpl = getDOMImplementation()
        adom = domimpl.createDocument(None, "start", None)
        robot_tag = adom.createElement("robot")
        for link in self.flat_links:
            robot_tag.appendChild(link.to_link_element(adom))
        first = True
        for link in self.flat_links:
            if first:# skip the root node!
                first = False
                continue
            robot_tag.appendChild(link.to_joint_element(adom))
        robot_tag.setAttribute("name", "pepe") #  choose a name!
        return '<?xml version="1.0"?>' + robot_tag.toprettyxml()

    def get_motors(self):
```

```
        self.get_flat_links2()
        if self.motors == None:
            motors = []
            for i in range(1, len(self.flat_links)):
                l = self.flat_links[i]
                m = creature.Motor(l.control_waveform, l.control_amp,    l.control_freq)
                motors.append(m)
            self.motors = motors
        return self.motors

def update_position(self, pos):
        if self.start_position == None:
            self.start_position = pos
        else:
            self.last_position = pos

def get_distance_travelled0(self):
        if self.start_position is None or self.last_position is None:
            return 0
        p1 = np.asarray(self.start_position)
        p2 = np.asarray(self.last_position)
        dist = np.linalg.norm(p1-p2)
        return dist



def get_distance_travelled2(self):
        if self.start_position is None or self.last_position is None:
            return 0
```

```
            p1 = np.asarray([0, 0, 5.0])
            p2 = np.asarray(self.last_position)
            THRESHOLD_DISTANCE = 20.0
            dist0 = np.linalg.norm(p1-p2)
            dist = (THRESHOLD_DISTANCE-dist0) if dist0 <= THRESHOLD_DISTANCE else 0
            return dist

        def get_distance_travelled(self):
            return self.get_distance_travelled2()

        def update_dna2(self, dna):
            self.dna = dna
            self.flat_links = None
            self.exp_links = None
            self.motors = None
            self.start_position = None
            self.last_position = None
```

<genome_wit_fixed_body.py>

```
import numpy as np
import copy
import random
import genome

#genome class with encoding scheme
class Genome_with_fixed_body():
    @staticmethod
    def get_random_gene2(length):
```

```
        gene = np.array([np.random.random() for i in range(length)])
        return gene

    @staticmethod
    def get_random_genome2(gene_count=1):
        gene_length=len(Genome_with_fixed_body.get_gene_spec2())
        genome = [Genome_with_fixed_body.get_random_gene2(gene_length) for i in range(gene_count)]
        return genome

    @staticmethod
    def get_gene_spec2():
        gene_spec =   {
            "link-shape":{"scale":1},
            "link-length": {"scale":2},
            "link-radius": {"scale":1},
# no use for link recurrence in new class
            #fixed "link-recurrence": {"scale":3},
            "link-mass": {"scale":1},
            "joint-type": {"scale":1},
# fixed parent for new class
            #fixed "joint-parent":{"scale":1},
            "joint-axis-xyz": {"scale":1},
            "joint-origin-rpy-1":{"scale":np.pi * 2},
            "joint-origin-rpy-2":{"scale":np.pi * 2},
            "joint-origin-rpy-3":{"scale":np.pi * 2},
            "joint-origin-xyz-1":{"scale":1},
            "joint-origin-xyz-2":{"scale":1},
            "joint-origin-xyz-3":{"scale":1},
```

```
                "control-waveform":{"scale":1},
                "control-amp":{"scale":0.25},
                "control-freq":{"scale":1}
                }
        ind = 0
        for key in gene_spec.keys():
                gene_spec[key]["ind"] = ind
                ind = ind + 1
        return gene_spec

    @staticmethod
    def get_gene_dict2(gene, spec):
        gdict = {}
        for key in spec:
                ind = spec[key]["ind"]
                scale = spec[key]["scale"]
                gdict[key] = gene[ind] * scale
        return gdict

    @staticmethod
    def get_genome_dicts2(genome, spec):
        gdicts = []
        for gene in genome:
                gdicts.append(Genome_with_fixed_body.get_gene_dict2(gene, spec))
        return gdicts

#fixed one paranet with new class
    @staticmethod
```

```
def get_Parent_Link(link_name="0"):
    link = genome.URDFLink(name=link_name,
                parent_name=link_name,
                recur=0, #recur+1,
                link_length=1.0, #fixed gdict["link-length"],
                link_radius=0.1, #fixed gdict["link-radius"],
                link_mass=1.4, #fixed gdict["link-mass"],
                joint_type=2, #gdict["joint-type"],
                joint_parent=0, #fixed gdict["joint-parent"],
                joint_axis_xyz=2, #gdict["joint-axis-xyz"],
                joint_origin_rpy_1=2, #gdict["joint-origin-rpy-1"],
                joint_origin_rpy_2=2, #gdict["joint-origin-rpy-2"],
                joint_origin_rpy_3=2, #gdict["joint-origin-rpy-3"],
                joint_origin_xyz_1=2, #gdict["joint-origin-xyz-1"],
                joint_origin_xyz_2=2, #gdict["joint-origin-xyz-2"],
                joint_origin_xyz_3=2, #gdict["joint-origin-xyz-3"],
                control_waveform=2, #gdict["control-waveform"],
                control_amp=2, #gdict["control-amp"],
                control_freq=2 #gdict["control-freq"])
    )
    return link


@staticmethod
def genome_to_flat_links2(gdicts):
    links = []
    link_ind = 0
    parent_names = [str(link_ind)]
    links.append(Genome_with_fixed_body.get_Parent_Link())
```

```
            link_ind = link_ind + 1
            for gdict in gdicts:
                link_name = str(link_ind)
                parent_ind = 0 #gdict["joint-parent"] * len(parent_names)
                assert parent_ind < len(parent_names), "genome.py: parent ind too high: " + str(parent_ind)
+ "got: " + str(parent_names)
                parent_name = parent_names[int(parent_ind)]
                #print("available parents: ", parent_names, "chose", parent_name)
                recur = 2 #gdict["link-recurrence"]
                link = genome.URDFLink(name=link_name,
                                       parent_name=parent_name,
                                       recur=recur+1,
#fixed shape
                                       link_length=0.01, #fixed gdict["link-length"],
                                       link_radius=0.30, #fixed gdict["link-radius"],
                                       link_mass=0.2, #fixed gdict["link-mass"],
                                       joint_type=gdict["joint-type"],
#fixed parent
                                       joint_parent=0, #fixed gdict["joint-parent"],
                                       joint_axis_xyz=gdict["joint-axis-xyz"],
                                       joint_origin_rpy_1=gdict["joint-origin-rpy-1"],
                                       joint_origin_rpy_2=gdict["joint-origin-rpy-2"],
                                       joint_origin_rpy_3=gdict["joint-origin-rpy-3"],
                                       joint_origin_xyz_1=gdict["joint-origin-xyz-1"],
                                       joint_origin_xyz_2=gdict["joint-origin-xyz-2"],
                                       joint_origin_xyz_3=gdict["joint-origin-xyz-3"],
                                       control_waveform=gdict["control-waveform"],
                                       control_amp=gdict["control-amp"],
                                       control_freq=gdict["control-freq"])
```

```
            links.append(link)
            if link_ind != 0:# don't re-add the first link
                parent_names.append(link_name)
            link_ind = link_ind + 1

        # now just fix the first link so it links to nothing
        links[0].parent_name = "None"
        return links

    @staticmethod
    def expandLinks2(parent_link, uniq_parent_name, flat_links, exp_links):
        children = [l for l in flat_links if l.parent_name == parent_link.name]
        sibling_ind = 1
        for c in children:
            for r in range(int(c.recur)):
                sibling_ind   = sibling_ind +1
                c_copy = copy.copy(c)
                c_copy.parent_name = uniq_parent_name
                uniq_name = c_copy.name + str(len(exp_links))
                #print("exp: ", c.name, " -> ", uniq_name)
                c_copy.name = uniq_name
                c_copy.sibling_ind = sibling_ind
                exp_links.append(c_copy)
                assert c.parent_name != c.name, "Genome::expandLinks: link joined to itself: " + c.name
+ " joins " + c.parent_name
                Genome.expandLinks(c, uniq_name, flat_links, exp_links)
```

```
@staticmethod
def crossover2(g1, g2):
    x1 = random.randint(0, len(g1)-1)
    x2 = random.randint(0, len(g2)-1)
    g3 = np.concatenate((g1[x1:], g2[x2:]))
    if len(g3) > len(g1):
        g3 = g3[0:len(g1)]
    return g3

@staticmethod
def point_mutate2(genome, rate, amount):
    new_genome = copy.copy(genome)
    for gene in new_genome:
        for i in range(len(gene)):
            if random.random() < rate:
                gene[i] += 0.1
            if gene[i] >= 1.0:
                gene[i] = 0.9999
            if gene[i] < 0.0:
                gene[i] = 0.0
    return new_genome

@staticmethod
def shrink_mutate2(genome, rate):
    if len(genome) == 1:
        return copy.copy(genome)
    if random.random() < rate:
        ind = random.randint(0, len(genome)-1)
```

```
            new_genome = np.delete(genome, ind, 0)
            return new_genome
        else:
            return copy.copy(genome)

@staticmethod
def grow_mutate2(genome, rate):
        if random.random() < rate:
            gene = Genome_with_fixed_body.get_random_gene2(len(genome[0]))
            new_genome = copy.copy(genome)
            new_genome = np.append(new_genome, [gene], axis=0)
            return new_genome
        else:
            return copy.copy(genome)



@staticmethod
def to_csv2(dna, csv_file):
        csv_str = ""
        for gene in dna:
            for val in gene:
                csv_str = csv_str + str(val) + ","
            csv_str = csv_str + '₩n'

        with open(csv_file, 'w') as f:
            f.write(csv_str)

@staticmethod
```

```
    def from_csv2(filename):
        csv_str = ''
        with open(filename) as f:
            csv_str = f.read()
        dna = []
        lines = csv_str.split('₩n')
        for line in lines:
            vals = line.split(',')
            gene = [float(v) for v in vals if v != '']
            if len(gene) > 0:
                dna.append(gene)
        return dna
```

<population_with_fixed_body.py>

```
import creature_with_fixed_body
import numpy as np

class Population_with_fixed_body:
    def __init__(self, pop_size, gene_count):
        self.creatures = [creature_with_fixed_body.Creature_with_fixed_body(
                                gene_count=gene_count)
                                for i in range(pop_size)]

    @staticmethod
    def get_fitness_map2(fits):
        fitmap = []
```

```
        total = 0
        for f in fits:
            total = total + f
            fitmap.append(total)
        return fitmap

    @staticmethod
    def select_parent2(fitmap):
        r = np.random.rand() # 0-1
        r = r * fitmap[-1]
        for i in range(len(fitmap)):
            if r <= fitmap[i]:
                return i
```

<simulation.py>

```
import pybullet as p
from multiprocessing import Pool

class Simulation:
    def __init__(self, sim_id=0):
        self.physicsClientId = p.connect(p.DIRECT)
        self.sim_id = sim_id

    def run_creature_original(self, cr, iterations=2400):
        pid = self.physicsClientId
        p.resetSimulation(physicsClientId=pid)
        p.setPhysicsEngineParameter(enableFileCaching=0, physicsClientId=pid)
```

```
        p.setGravity(0, 0, -10, physicsClientId=pid)
        plane_shape = p.createCollisionShape(p.GEOM_PLANE, physicsClientId=pid)
        floor = p.createMultiBody(plane_shape, plane_shape, physicsClientId=pid)

        xml_file = 'temp' + str(self.sim_id) + '.urdf'
        xml_str = cr.to_xml()
        with open(xml_file, 'w') as f:
            f.write(xml_str)

        cid = p.loadURDF(xml_file, physicsClientId=pid)

        p.resetBasePositionAndOrientation(cid, [0, 0, 2.5], [0, 0, 0, 1], physicsClientId=pid)


        for step in range(iterations):
            p.stepSimulation(physicsClientId=pid)
            if step % 24 == 0:
                self.update_motors(cid=cid, cr=cr)

            pos, orn = p.getBasePositionAndOrientation(cid, physicsClientId=pid)
            cr.update_position(pos)
            #print(pos[2])
            #print(cr.get_distance_travelled())

    def set_as_GUI(self):
        self.physicsClientId = p.connect(p.GUI)
```

```
    def make_arena(self, arena_size=10, wall_height=1):
        wall_thickness = 0.5
        floor_collision_shape = p.createCollisionShape(shapeType=p.GEOM_BOX, halfExtents=[arena_size/2, arena_size/2, wall_thickness], physicsClientId=self.physicsClientId)
        floor_visual_shape = p.createVisualShape(shapeType=p.GEOM_BOX, halfExtents=[arena_size/2, arena_size/2, wall_thickness], rgbaColor=[1, 1, 0, 1], physicsClientId=self.physicsClientId)
        floor_body = p.createMultiBody(baseMass=0, baseCollisionShapeIndex=floor_collision_shape, baseVisualShapeIndex=floor_visual_shape, basePosition=[0, 0, -wall_thickness], physicsClientId=self.physicsClientId)

        wall_collision_shape = p.createCollisionShape(shapeType=p.GEOM_BOX, halfExtents=[arena_size/2, wall_thickness/2, wall_height/2], physicsClientId=self.physicsClientId)
        wall_visual_shape = p.createVisualShape(shapeType=p.GEOM_BOX, halfExtents=[arena_size/2, wall_thickness/2, wall_height/2], rgbaColor=[0.7, 0.7, 0.7, 1], physicsClientId=self.physicsClientId)  # Gray walls

        # Create four walls
        p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape, baseVisualShapeIndex=wall_visual_shape, basePosition=[0, arena_size/2, wall_height/2], physicsClientId=self.physicsClientId)
        p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape, baseVisualShapeIndex=wall_visual_shape, basePosition=[0, -arena_size/2, wall_height/2], physicsClientId=self.physicsClientId)

        wall_collision_shape = p.createCollisionShape(shapeType=p.GEOM_BOX, halfExtents=[wall_thickness/2, arena_size/2, wall_height/2], physicsClientId=self.physicsClientId)
        wall_visual_shape = p.createVisualShape(shapeType=p.GEOM_BOX, halfExtents=[wall_thickness/2, arena_size/2, wall_height/2], rgbaColor=[0.7, 0.7, 0.7, 1], physicsClientId=self.physicsClientId)  # Gray walls

        p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape, baseVisualShapeIndex=wall_visual_shape, basePosition=[arena_size/2, 0, wall_height/2], physicsClientId=self.physicsClientId)
        p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape,
```

```
baseVisualShapeIndex=wall_visual_shape, basePosition=[-arena_size/2, 0, wall_height/2], physicsClientId=self.physicsClientId)


    def make_mountain(self):
        mountain_position = (0, 0, -1)  # Adjust as needed
        mountain_orientation = p.getQuaternionFromEuler((0, 0, 0))
        p.setAdditionalSearchPath('shapes/')
        # mountain = p.loadURDF("mountain.urdf", mountain_position, mountain_orientation, useFixedBase=1)
        # mountain = p.loadURDF("mountain_with_cubes.urdf", mountain_position, mountain_orientation, useFixedBase=1)

        mountain = p.loadURDF("gaussian_pyramid.urdf", mountain_position, mountain_orientation, useFixedBase=1, physicsClientId=self.physicsClientId)

#run one creature in the new arena
    def run_one_creature_in_the_arena(self, cr, motor_update_count=2400):
        if cr == None:
            print("No creatue. so exit")
            return
        pid = self.physicsClientId
        p.resetSimulation(physicsClientId=pid)
        p.setPhysicsEngineParameter(enableFileCaching=0, physicsClientId=pid)

        p.resetDebugVisualizerCamera(
            cameraDistance=20,
            cameraYaw=50,
            cameraPitch=-35,
            cameraTargetPosition=[0, 0, 0],
```

```
            physicsClientId=pid
        )

        p.setGravity(0, 0, -10, physicsClientId=pid)
        # plane_shape = p.createCollisionShape(p.GEOM_PLANE, physicsClientId=pid)
        # floor = p.createMultiBody(plane_shape, plane_shape, physicsClientId=pid)

        self.make_arena(arena_size=20)
        self.make_mountain()
        xml_file = 'temp' + str(self.sim_id) + '.urdf'
        xml_str = cr.to_xml()
        with open(xml_file, 'w') as f:
            f.write(xml_str)

        cid = p.loadURDF(xml_file, physicsClientId=pid)

#             p.resetBasePositionAndOrientation(cid, [0, 0, 2.5], [0, 0, 0, 1], physicsClientId=pid)
        p.resetBasePositionAndOrientation(cid, [5, 5, 2.5], [0, 0, 0, 1], physicsClientId=pid)

        p.setRealTimeSimulation(1, physicsClientId=pid)

        for step in range(motor_update_count):
            p.stepSimulation(physicsClientId=pid)
            if step % 24 == 0:
                self.update_motors(cid=cid, cr=cr)

            pos, orn = p.getBasePositionAndOrientation(cid, physicsClientId=pid)
            cr.update_position(pos)
```

```
            #print(pos[2])
            #print(cr.get_distance_travelled())
        if p.GUI == p.getConnectionInfo(physicsClientId=pid)['connectionMethod']:
            print(cr.get_distance_travelled())




def update_motors(self, cid, cr):
    """
    cid is the id in the physics engine
    cr is a creature object
    """
    for jid in range(p.getNumJoints(cid,
                                    physicsClientId=self.physicsClientId)):
        m = cr.get_motors()[jid]

        p.setJointMotorControl2(cid, jid,
                    controlMode=p.VELOCITY_CONTROL,
                    targetVelocity=m.get_output(),
                    force = 5,
                    physicsClientId=self.physicsClientId)



# You can add this to the Simulation class:
def eval_population(self, pop, iterations):
    for cr in pop.creatures:
```

```
            self.run_creature(cr, 2400)

    def eval_population_in_the_arena(self, pop, iterations):
        for cr in pop.creatures:
            self.run_one_creature_in_the_arena(cr, 2400)

class ThreadedSim():
    def __init__(self, pool_size):
        self.sims = [Simulation(i) for i in range(pool_size)]

    @staticmethod
    def static_run_creature(sim, cr, iterations):
        sim.run_creature(cr, iterations)
        return cr

    @staticmethod
    def static_run_creature_in_the_arena(sim, cr, iterations):
        sim.run_one_creature_in_the_arena(cr, iterations)
        return cr



    def eval_population(self, pop, iterations):
        """
        pop is a Population object
        iterations is frames in pybullet to run for at 240fps
        """
        pool_args = []
        start_ind = 0
```

```
        pool_size = len(self.sims)
        while start_ind < len(pop.creatures):
            this_pool_args = []
            for i in range(start_ind, start_ind + pool_size):
                if i == len(pop.creatures):# the end
                    break
                # work out the sim ind
                sim_ind = i % len(self.sims)
                this_pool_args.append([
                                self.sims[sim_ind],
                                pop.creatures[i],
                                iterations]
                )
            pool_args.append(this_pool_args)
            start_ind = start_ind + pool_size

        new_creatures = []
        for pool_argset in pool_args:
            with Pool(pool_size) as p:
                # it works on a copy of the creatures, so receive them
                creatures = p.starmap(ThreadedSim.static_run_creature, pool_argset)
                # and now put those creatures back into the main
                # self.creatures array
                new_creatures.extend(creatures)
        pop.creatures = new_creatures

    def eval_population_in_the_arena(self, pop, iterations):
        """
```

```
pop is a Population object
iterations is frames in pybullet to run for at 240fps
"""
pool_args = []
start_ind = 0
pool_size = len(self.sims)
while start_ind < len(pop.creatures):
    this_pool_args = []
    for i in range(start_ind, start_ind + pool_size):
        if i == len(pop.creatures):# the end
            break
        # work out the sim ind
        sim_ind = i % len(self.sims)
        this_pool_args.append([
                    self.sims[sim_ind],
                    pop.creatures[i],
                    iterations]
        )
    pool_args.append(this_pool_args)
    start_ind = start_ind + pool_size

new_creatures = []
for pool_argset in pool_args:
    with Pool(pool_size) as p:
        # it works on a copy of the creatures, so receive them
        creatures = p.starmap(ThreadedSim.static_run_creature_in_the_arena, pool_argset)
        # and now put those creatures back into the main
        # self.creatures array
```

```
                new_creatures.extend(creatures)

    pop.creatures = new_creatures
```